

벼리 · Byeori

Byeori — The Thread of K-Tradition

개발 표준 기술 설계 문서

명명 규칙 · 파일 구조 · API 명세 · 인프라 Setup

한국 전통 테마 관광 큐레이션 플랫폼

2026 관광데이터 활용 공모전 — 웹·앱 개발 부문

문서 버전 v1.0 · 작성일 2026-06-01

기준 ERD erd_final_1.txt (v3)

목차

1. 명명 규칙 (Naming Conventions)

- 1.1 표기법 계층 개요
 - 1.2 데이터베이스 (PostgreSQL)
 - 1.3 enum · 상태값 표준 (전수)
 - 1.4 백엔드 (Java / Spring)
 - 1.5 REST API
 - 1.6 프론트엔드 (앱 · 웹)
 - 1.7 Git · GitHub 협업 규칙
-

2. 파일 구조 (File Structure)

- 2.1 모노레포 전체
 - 2.2 백엔드 (by-feature)
 - 2.3 앱 (Expo React Native)
 - 2.4 웹 (Next.js 어드민)
 - 2.5 인프라
-

3. API 명세서 (API Specification)

- 3.1 공통 규약
 - 3.2 다형성 타겟 (targetType / targetId)
 - 3.3 도메인별 엔드포인트
 - 3.4 어드민 API
 - 3.5 배치 (비공개)
-

4. 인프라 Setup (Infrastructure)

- 4.1 구성 개요
- 4.2 docker-compose
- 4.3 nginx · TLS
- 4.4 환경변수
- 4.5 스케줄 배치
- 4.6 CI/CD

- 4.7 백업 · 운영
 - 4.8 공공데이터 이용 의무 체크리스트
 - 4.9 로컬 개발 시작
-

문서 개요

벼리(Byeori) 는 흩어져 있는 한국 전통 테마 매장·코스 정보를 한곳에 엮어 내·외국인 관광객에게 명확한 관광 동선을 제시하는 다국어 큐레이션 플랫폼이다. 한복 착용 시 혜택 매장 필터, 운영진 큐레이션 추천 코스, 사용자 여행 일지, KOPIS·TourAPI·카카오맵 연동을 핵심으로 한다.

본 문서는 ERD(`erd_final_1.txt`)를 **단일 진실 원천(SSOT)** 으로 삼아 개발에 필요한 4가지 표준을 정의한다.

구분	결정
백엔드	Spring Boot (Java) + JPA / QueryDSL
데이터베이스	PostgreSQL (부분 인덱스 · CHECK 제약 사용)
클라이언트	React Native(Expo) 앱 + Next.js 운영자 어드민 웹
서버 상태 / 전역 상태	TanStack Query + zustand
인프라	Docker Compose + 단일 VM
저장소 구성	모노레포
DB 마이그레이션	Flyway (api 부팅 시 자동 실행)
이미지 저장	MinIO (S3 호환) · KOPIS 포스터는 외부 URL 참조

v3 ERD 변경 반영: 어드민 권한 식별을 위한 `users.role` (USER/ADMIN), 한국관광공사 OpenAPI 수집 데이터 식별을 위한 `source=TOURAPI` 및 `tour_content_id` , `sync_logs.provider` 추가.

1. 명명 규칙

1.1 표기법 계층 개요

계층마다 표기법이 다르며, 경계(JPA · API 직렬화)에서 한 번씩 변환한다.

계층	표기법	예시
데이터베이스 (테이블 · 컬럼)	snake_case	hanbok_discount , created_by_user_id
Java 클래스 · 인터페이스 · enum	PascalCase	Venue , VenueService , Source
Java 필드 · 메서드 · 지역변수	camelCase	hanbokDiscount , findByVenueId()
Java 상수 (static final)	UPPER_SNAKE	MAX_PAGE_SIZE
API JSON 바디 · 쿼리 파라미터	camelCase	{ "hanbokDiscount": true } , ?targetType=
프론트 변수 · 함수	camelCase	fetchVenues
환경변수	UPPER_SNAKE	KOPIS_API_KEY

DB ↔ Java 변환은 JPA 네이밍 전략 CamelCaseToUnderscoresNamingStrategy (Spring Boot 기본)가 처리하므로 엔티티에 @Column(name=...) 을 일일이 붙이지 않는다.

1.2 데이터베이스 (PostgreSQL)

대상	규칙	예시
테이블	snake_case 복수형	users , venues , curated_courses
컬럼	snake_case	start_date , external_booking_url
기본키	id (bigint , identity)	id
외래키	<단수엔티티>_id	venue_id , user_id , created_by_user_id
일반 인덱스	ix_<table>_<cols>	ix_performances_start_date
유니크 인덱스	ux_<table>_<cols>	ux_venues_kopis_id
부분 유니크	... WHERE <col> IS NOT NULL	ux_venues_kopis_id WHERE kopis_id IS NOT NULL
불리언	의미형 (조동사/형용사)	hanbok_discount , is_required , phone_verified
시각	*_at (timestamp)	created_at , withdrawn_at , synced_at
금액·평점	decimal(p,s)	extra_price decimal(10,2) , avg_rating decimal(2,1)

다형성 참조 처리 — wishlists , reviews , content_tag_votes , curated_course_items , itinerary_items 는 performance_id / venue_id 두 컬럼 중 정확히 하나만 채운다.

```
-- 모든 다형성 테이블 공통
CHECK ( (performance_id IS NULL) <> (venue_id IS NULL) )
-- NULL 포함 유니크는 조건부 인덱스로 (예: wishlists)
CREATE UNIQUE INDEX ux_wishlists_user_perf ON wishlists(user_id, performance_id) WHERE
performance_id IS NOT NULL;
CREATE UNIQUE INDEX ux_wishlists_user_venue ON wishlists(user_id, venue_id) WHERE
venue_id IS NOT NULL;
```

1.3 enum · 상태값 표준 (전수)

원칙 — 상태·타입·플래그성 코드값은 UPPER_SNAKE 영문 고정. 사용자 노출 분류값(category , interest , theme , reason)은 한국어 라벨을 허용하되 코드화가 필요하면 별도 매핑 테이블/enum으로 승격한다.

컬럼	허용값
users.auth_provider	LOCAL KAKAO NAVER APPLE GOOGLE
users.role	USER ADMIN
users.status	ACTIVE WITHDRAWN
users.language	ko en ja zh
social_auths.provider	KAKAO NAVER GOOGLE APPLE
user_interests.category	한복 · 음식 · 체험 · 문화 · 공연 (라벨)
terms.type	SERVICE PRIVACY MARKETING
venues.source	KOPIS TOURAPI MANUAL USER
venues.category	한복 · 메이크업 · 체험 · 카페 · 맛집 · 음식점 ... (라벨)
venues.visibility	PUBLIC PRIVATE
venues.status	ACTIVE INACTIVE
venue_reports.reason	허위정보 · 폐업 · 부적절 ... (라벨)
venue_reports.status	PENDING REVIEWED DISMISSED
performances.state	UPCOMING ONGOING ENDED
performances.source	KOPIS TOURAPI MANUAL
performance_medias.media_type	IMAGE VIDEO
performance_schedules.status	AVAILABLE CANCELLED
reviews.rating	1 ~ 5 (정수)
comment_tags.status	ACTIVE INACTIVE
curated_courses.theme	한복나들이 · 미식투어 · 반나절코스 ... (라벨)
curated_courses.status	ACTIVE INACTIVE
itineraries.source_type	CUSTOM CURATED
sync_logs.provider	KOPIS TOURAPI
sync_logs.target_type	PERFORMANCE VENUE
sync_logs.status	SUCCESS PARTIAL FAILED

1.4 백엔드 (Java / Spring)

대상	규칙	예시
패키지	전부 소문자, <code>com.byeori.<domain></code>	<code>com.byeori.venue</code>
엔티티 클래스	도메인 단수 (접미사 없음)	<code>Venue</code> , <code>Performance</code>
리포지토리	<code><Entity>Repository</code>	<code>VenueRepository</code>
서비스	<code><Entity>Service</code>	<code>VenueService</code>
컨트롤러	<code><Entity>Controller</code> / <code><Entity>AdminController</code>	<code>VenueController</code>
요청 DTO	<code><동작><Entity>Request</code>	<code>VenueCreateRequest</code> , <code>ReviewUpdateRequest</code>
응답 DTO	<code><Entity>Response</code> , <code><Entity>DetailResponse</code>	<code>VenueResponse</code>
enum	명사 단수	<code>Source</code> , <code>VenueStatus</code> , <code>ReportStatus</code>
상수	<code>UPPER_SNAKE</code>	<code>DEFAULT_PAGE_SIZE</code>
메서드	<code>camelCase</code> , 동사 시작	<code>findById</code> , <code>recalcRating</code>

리포지토리 쿼리 메서드는 Spring Data 규칙(`findBy...` , `existsBy...` , `countBy...`)을 따른다.

1.5 REST API

- 베이스: `/api/v1`
- 어드민: `/api/v1/admin/...` (별도 prefix)
- 리소스는 복수 명사, 경로변수는 `{id}` , 하위 리소스는 중첩(`/itineraries/{id}/items`)
- 쿼리 파라미터·JSON 키는 `camelCase` (`?hanbokDiscount=true` , `?targetType=VENUE`)
- 페이지네이션: `page` , `size` , `sort` (예: `sort=createdAt,desc`)

```
GET    /api/v1/venues?category=한복&hanbokDiscount=true&page=0&size=20
POST   /api/v1/reviews          { "targetType": "PERFORMANCE", "targetId": 42, "rating": 5 }
DELETE /api/v1/itineraries/{id}/items/{itemId}
```

1.6 프론트엔드 (앱 · 웹)

두 클라이언트(Expo RN, Next.js) 공통 TypeScript 규칙.

파일 명명

종류	규칙	예시
컴포넌트	<code>PascalCase.tsx</code>	<code>VenueCard.tsx</code> , <code>HanbokFilterToggle.tsx</code>
라우트 파일	라우터 규약(소문자/특수파일)	RN <code>app/venues/[id].tsx</code> · Next <code>app/venues/page.tsx</code>
커스텀 훅	<code>useXxx.ts</code>	<code>useVenues.ts</code> , <code>useAuth.ts</code>
API 모듈	<code>camelCase.ts</code>	<code>venues.ts</code> , <code>client.ts</code>
타입	feature별 <code>types.ts</code>	<code>features/venue/types.ts</code>
상수	<code>constants.ts</code> (값 <code>UPPER_SNAKE</code>)	<code>PAGE_SIZE</code>
i18n 로케일	언어코드	<code>ko.json</code> <code>en.json</code> <code>ja.json</code> <code>zh.json</code>
테스트	<code><대상>.test.tsx</code>	<code>VenueCard.test.tsx</code>

함수 · 변수 명명

종류	규칙	예시
컴포넌트	<code>PascalCase</code> 함수	<code>function VenueCard() {}</code>
이벤트 핸들러	<code>handleXxx</code>	<code>handleSubmit</code> , <code>handlePressVenue</code>
API 호출 함수	동사 시작	<code>fetchVenues</code> , <code>createReview</code> , <code>deleteWishlist</code>
불리언	<code>is</code> / <code>has</code> / <code>should</code>	<code>isLoading</code> , <code>hasNextPage</code>
타입 · 인터페이스	<code>PascalCase</code>	<code>Venue</code> , <code>VenueListResponse</code>
i18n 키	dot 네임스페이스	<code>venue.detail.title</code> , <code>common.button.save</code>

TanStack Query · zustand 훅 규칙

종류	규칙	예시
조회 훅	<code>use<Entity>Query</code> / <code>use<Entity>DetailQuery</code>	<code>useVenuesQuery</code> , <code>useVenueDetailQuery</code>
변경 훅	<code>useCreate/Update/Delete<Entity>Mutation</code>	<code>useCreateReviewMutation</code>
zustand 스토어	<code>use<Name>Store</code>	<code>useAuthStore</code> , <code>useLocaleStore</code>
queryKey	배열, [리소스, 식별자/파라미터]	<code>['venues', params]</code> , <code>['venue', id]</code>

1.7 Git · GitHub 협업 규칙

브랜치 전략 (GitHub Flow)

`main` 은 항상 배포 가능한 상태를 유지하는 보호 브랜치다. 모든 작업은 짧은 수명의 토픽 브랜치에서 진행하고 **Pull Request** → 리뷰 → **squash** 머지로 합친다. `main` 에 머지(push)되면 CI가 자동 배포한다(§ 4.6).



- `main` 직접 push 금지 (브랜치 보호 규칙 설정 권장)
- 운영 긴급 수정은 `hotfix/` 를 `main` 에서 분기

브랜치 명명 규칙

형식: `<type>/<이슈번호>-<요약>` — 소문자 **kebab-case**, 영문, 요약 2~4단어. 이슈번호는 있으면 앞에 붙인다(없으면 생략).

```
feature/12-hanbok-filter
fix/45-review-rating-cache
docs/branch-convention
chore/seed-tour-api-sync
hotfix/89-login-token-expiry
```

type	용도	분기 기준
<code>feature/</code>	기능 개발	<code>main</code>
<code>fix/</code>	버그 수정	<code>main</code>
<code>hotfix/</code>	운영 긴급 수정	<code>main</code>
<code>refactor/</code>	동작 변경 없는 구조 개선	<code>main</code>
<code>chore/</code>	설정·빌드·의존성·시드	<code>main</code>
<code>docs/</code>	문서	<code>main</code>
<code>test/</code>	테스트 추가·보강	<code>main</code>

규칙: 공백·대문자·한글 금지(영문 kebab), 단어 구분은 `-`, type과 요약 구분은 `/`. 개인 식별이 필요하면 `<type>/<이름>/<요약>` 도 허용.

커밋 메시지 (Conventional Commits)

형식: `<type>(<scope>): <요약>` — 모노레포이므로 `scope` 로 영역을 명시한다.

- type : feat fix refactor chore docs test style perf

- `scope` : `api` `app` `web` `infra` `docs` (생략 가능)

`feat(api)`: 한복 혜택 필터 쿼리 추가
`fix(app)`: 리뷰 평점 캐시 미갱신 수정
`docs`: 브랜치 명명 규칙 추가

Pull Request · 머지

- PR 제목도 Conventional Commits 형식을 따른다.
- 머지 전략은 **Squash and merge** (커밋 히스토리 단정하게 유지).
- 머지 후 토픽 브랜치는 **삭제**한다.
- 최소 1인 리뷰 승인 후 머지(브랜치 보호 규칙).

환경변수

`UPPER_SNAKE` (`JWT_ACCESS_SECRET` , `KAKAO_REST_API_KEY`). 상세는 § 4.4.

2. 파일 구조

2.1 모노레포 전체

```
byeori/  
├ apps/  
│ ├── api/          # Spring Boot (백엔드)  
│ ├── app/          # Expo React Native (모바일 앱)  
│ └── web/           # Next.js (운영자 어드민)  
├ infra/            # docker-compose, nginx, env, scripts  
├ docs/             # 본 문서 등 산출물  
└ .github/workflows/ # CI/CD
```

2.2 백엔드 (by-feature)

도메인 폴더마다 `controller / service / repository / entity / dto` 가 반복된다. 위성 도메인 (review·wishlist·tag·course·itinerary)은 코어 도메인(venue·performance)을 **단방향으로만** 의존한다.

```
apps/api/  
├ build.gradle  
└ src/main/  
  ├── java/com/byeori/  
  │ ├── domain/  
  │ │ ├── user/      controller/ service/ repository/ entity/ dto/  
  │ │ ├── auth/      # 로컬·소셜 로그인, JWT 발급, 약관 동의  
  │ │ ├── venue/     # 장소 + venue_reports + USER 등록 장소  
  │ │ ├── performance/ # 공연 + medias/options/schedules  
  │ │ ├── review/  
  │ │ ├── wishlist/  
  │ │ ├── tag/       # comment_tags + content_tag_votes  
  │ │ ├── course/    # curated_courses + items  
  │ │ ├── itinerary/ # itineraries + items  
  │ │ └── sync/      # KOPIS·TourAPI 동기화 배치 + sync_logs  
  │ └── global/  
  │ ├── config/      # Security, Jackson, Web, Scheduler  
  │ ├── security/    # JWT 필터, ROLE_ADMIN 가드  
  │ ├── exception/   # 전역 핸들러 + 에러코드  
  │ ├── response/    # 공통 응답 봉투 ApiResponse  
  │ ├── content/     # ContentType 값객체 (targetType↔컬럼 변환)  
  │ └── external/    # KopisClient, TourApiClient, KakaoMapClient  
  └ resources/  
    ├── application.yml (+ -dev.yml / -prod.yml)  
    └ db/migration/    # Flyway: V1__init.sql, V2__...
```

2.3 앱 (Expo React Native)

```
apps/app/
├─ app/                                # expo-router (파일 기반 라우팅)
│  ├─ (tabs)/  index.tsx courses.tsx itinerary.tsx profile.tsx
│  ├─ venues/[id].tsx
│  └─ performances/[id].tsx
└─ src/
   ├─ api/          client.ts venues.ts performances.ts reviews.ts ...  # 봉투 언랩 · Accept-
Language 주입
   ├─ components/ # 공용 UI (Button, Card, Rating ...)
   ├─ features/
   │  ├─ venue/          components/ hooks/ types.ts
   │  └─ performance/  review/ wishlist/ course/ itinerary/ map/
   ├─ hooks/            # 전역 useAuth, useLocale
   ├─ store/            # zustand (authStore, localeStore)
   ├─ i18n/             index.ts locales/ ko.json en.json ja.json zh.json
   ├─ theme/            # 색·타이포·간격 토큰
   └─ utils/ constants/
```

2.4 웹 (Next.js 어드민)

```
apps/web/
├─ app/                                # app router (소문자 세그먼트)
│  ├─ layout.tsx  page.tsx
│  ├─ venues/  page.tsx [id]/page.tsx
│  ├─ reports/  page.tsx
│  ├─ courses/  page.tsx
│  └─ sync/     page.tsx
├─ components/
├─ features/   venue/ report/ course/ sync/ # 각 components/ hooks/ api.ts types.ts
├─ lib/api/    client.ts venues.ts         # fetch 래퍼(토큰·봉투 언랩)
└─ hooks/     types/ i18n/
```

2.5 인프라

```
infra/
├─ docker-compose.yml
├─ docker-compose.override.yml # 로컬 개발용
├─ nginx/  conf.d/byeori.conf
├─ env/    .env.example
└─ scripts/ backup.sh  deploy.sh
```

3. API 명세서

3.1 공통 규약

항목	규약
베이스 URL	/api/v1
인증	JWT — Authorization: Bearer <accessToken>
다국어	Accept-Language: ko en ja zh (콘텐츠 언어 결정)
페이지네이션	오프셋 — page (0-base) · size · sort
키 표기	camelCase (바디 · 쿼리)

응답 봉투 — 모든 응답은 공통 래퍼로 감싼다.

```
// 성공
{ "success": true, "data": { /* ... */ }, "error": null }
// 실패
{ "success": false, "data": null,
  "error": { "code": "REVIEW_NOT_FOUND", "message": "리뷰를 찾을 수 없습니다." } }
```

목록 응답(페이지) data 형식:

```
{ "content": [ /* ... */ ], "page": 0, "size": 20, "totalElements": 134, "totalPages": 7 }
```

에러 코드 체계 — <DOMAIN>_<REASON> (AUTH_INVALID_TOKEN , VENUE_NOT_FOUND , REVIEW_FORBIDDEN , VALIDATION_FAILED). HTTP 상태코드와 병행 사용(400/401/403/404/409/500).

소셜 로그인 흐름

```
앱이 provider SDK 로그인 → provider accessToken 획득
→ POST /api/v1/auth/social { "provider": "KAKAO", "accessToken": "..." }
→ 백엔드가 provider에 검증 → users/social_auths upsert → 우리 JWT(Access+Refresh) 발급
```

JWT refresh 전략 — Access(단기) + Refresh(장기). Refresh 토큰은 1차에서 stateless 검증(서명+만료)으로 처리하고, 강제 로그아웃/블랙리스트가 필요해지면 refresh_tokens 저장소(ERD 외)를 후속 도입한다.

3.2 다형성 타깃 (targetType / targetId)

찜·리뷰·태그 투표·코스 항목·일지 항목은 대상이 공연 또는 장소다. API는 항상 { targetType, targetId } 로 주고받고, 백엔드 ContentType 값객체가 DB의 performance_id / venue_id 컬럼으로 변환한다.

```
{ "targetType": "PERFORMANCE" | "VENUE", "targetId": 42 }
```

`targetType` 누락·둘 다 지정 등은 `VALIDATION_FAILED` 로 거절(DB CHECK 제약과 이중 방어).

3.3 도메인별 엔드포인트

표기:  비인증 ·  인증 필요. 경로는 `/api/v1` 생략.

인증 · 회원

메서드	경로	인증	설명
POST	<code>/auth/signup</code>		로컬 회원가입 (약관 동의 포함)
POST	<code>/auth/login</code>		로컬 로그인 → JWT
POST	<code>/auth/social</code>		소셜 로그인 <code>{provider, accessToken}</code>
POST	<code>/auth/token/refresh</code>		Refresh로 Access 재발급
POST	<code>/auth/logout</code>		로그아웃
GET	<code>/users/me</code>		내 정보
PATCH	<code>/users/me</code>		내 정보 수정 (name, phone, language)
DELETE	<code>/users/me</code>		회원 탈퇴 (soft delete → <code>status=WITHDRAWN</code>)
GET · PUT	<code>/users/me/interests</code>		관심사 조회 · 설정 (<code>user_interests</code>)
GET · DELETE	<code>/users/me/social-auths</code>		연결된 소셜 계정 조회 · 연동 해제
GET	<code>/terms</code>		약관 목록(유효본)
POST	<code>/users/me/terms</code>		약관 동의 기록 (<code>user_terms</code>)

장소 (venues)

메서드	경로	인 증	설명
GET	/venues		목록 · 지도. 필터: <code>category</code> <code>hanbokDiscount</code> <code>source</code> <code>bbox</code> (지도 영역) <code>keyword</code>
GET	/venues/{id}		상세 (KOPIS·TOURAPI면 출처 표기 데이터 포함)
GET	/venues/{id}/performances		그 시설의 공연 목록
GET	/venues/{id}/nearby		주변 음식점 — 카카오맵 실시간(DB 비저장)
POST	/venues		USER 등록 장소(거지맵) — <code>source=USER</code> , <code>visibility</code>
PATCH	/venues/{id}		본인 등록 장소 수정
DELETE	/venues/{id}		본인 등록 장소 삭제
POST	/venues/{id}/reports		신고 (<code>venue_reports</code>)

공연 (performances)

메서드	경로	인 증	설명
GET	/performances		목록. 필터: <code>state</code> <code>startDate</code> <code>genre</code> <code>venueId</code> <code>keyword</code>
GET	/performances/{id}		상세 — <code>medias</code> · <code>options</code> · <code>schedules</code> · 외부예매 링크 · 출처
GET	/performances/{id}/schedules		회차 목록

찜 · 리뷰 · 태그

메서드	경로	인 증	설명
GET	/users/me/wishlists		내 찜 목록 (targetType 필터)
POST	/wishlists		찜 추가 {targetType, targetId}
DELETE	/wishlists		찜 해제 {targetType, targetId}
GET	/reviews? targetType=&targetId=		대상별 리뷰 목록
GET	/users/me/reviews		내 리뷰 목록
POST	/reviews		리뷰 작성 {targetType, targetId, rating, content} → avg_rating 동기 갱신
PATCH	/reviews/{id}		리뷰 수정 → avg_rating 재계산
DELETE	/reviews/{id}		리뷰 삭제 → avg_rating 재계산
GET	/comment-tags		태그 마스터(활성)
GET	/content-tags? targetType=&targetId=		대상의 태그별 집계 ("분위기 좋음 N명")
POST	/content-tag-votes		태그 투표 {commentTagId, targetType, targetId}
DELETE	/content-tag-votes		태그 투표 취소

추천 코스 · 여행 일지

메서드	경로	인증	설명
GET	<code>/curated-courses</code>		추천 코스 목록 (필터: <code>theme</code>)
GET	<code>/curated-courses/{id}</code>		상세 — items 동선(<code>sort_order</code>)
GET	<code>/users/me/itineraries</code>		내 여행 일지 목록
POST	<code>/itineraries</code>		생성 — <code>CUSTOM</code> 또는 <code>CURATED</code> (추천코스 복사, <code>sourceCourseId</code>)
GET	<code>/itineraries/{id}</code>		일지 상세
PATCH	<code>/itineraries/{id}</code>		일지 수정
DELETE	<code>/itineraries/{id}</code>		일지 삭제
POST	<code>/itineraries/{id}/items</code>		항목 추가 { <code>targetType</code> , <code>targetId</code> , <code>visitDate</code> , <code>plannedTime</code> }
PATCH	<code>/itineraries/{id}/items/{itemId}</code>		항목 수정 (순서 <code>sortOrder</code> 포함)
DELETE	<code>/itineraries/{id}/items/{itemId}</code>		항목 삭제

코스 공유 = 별도 UGC 테이블 없이 클라이언트 OS 공유 시트로 외부 SNS 공유(공개 콘텐츠는 공유 URL/딥링크). 인앱 좋아요·댓글·해시태그 피드는 ERD 미포함 → 향후 확장.

3.4 어드민 API `/api/v1/admin/**` (ROLE_ADMIN)

운영자 웹(Next.js) 전용. `users.role = ADMIN` 가드.

메서드	경로	설명
POST · PATCH · DELETE	<code>/admin/venues</code> <code>/admin/venues/{id}</code>	MANUAL 장소 CRUD + 한복 혜택 태깅 (<code>hanbokDiscount</code>)
POST · PATCH · DELETE	<code>/admin/performances</code> <code>/admin/performances/{id}</code>	MANUAL 공연 CRUD (+ medias · options · schedules)
GET	<code>/admin/venue-reports</code>	신고 목록 (필터: <code>status</code>)
PATCH	<code>/admin/venue-reports/{id}</code>	신고 검수 (<code>PENDING</code> → <code>REVIEWED/DISMISSED</code>)
GET · POST · PATCH · DELETE	<code>/admin/comment-tags</code>	태그 마스터 관리
GET · POST · PATCH · DELETE	<code>/admin/curated-courses</code>	추천 코스 큐레이션 (+ items)
GET · POST · PATCH · DELETE	<code>/admin/terms</code>	약관 관리 (<code>effective_at</code> 버전)
GET	<code>/admin/sync-logs</code>	동기화 로그 (필터: <code>provider</code> <code>targetType</code> <code>status</code>)
POST	<code>/admin/sync/trigger</code>	수동 동기화 트리거 <code>{provider, targetType}</code>

3.5 배치 (비공개)

KOPIS·TourAPI 동기화는 백엔드 `@Scheduled` 배치로만 수행하며 **공개 엔드포인트가 없다**(인증키 백엔드 전용). 운영자는 `/admin/sync/trigger` 로 수동 실행, `/admin/sync-logs` 로 결과를 확인한다. 동기화 순서는 **반드시 시설(venue) → 공연(performance)** (FK NOT NULL).

4. 인프라 Setup

4.1 구성 개요

```

[ 단일 VM ]
인터넷 —443→ nginx (리버스 프록시 + TLS/certbot)
                |
                |— /api → byeori-api (Spring Boot :8080)
                |           |
                |           |— @Scheduled: KOPIS·TourAPI 동기화 / 탈퇴 영구삭제
                |           |
                |           |— / → byeori-web (Next.js :3000, 운영자 어드민)
                |
byeori-api → postgres (:5432, 볼륨 영속)
byeori-api → minio     (:9000, S3 호환 — MANUAL 업로드 이미지)
```

서비스: `nginx` · `byeori-api` · `byeori-web` · `postgres` · `minio` . **Redis 미도입**(avg_rating 동기 처리, 큐 불필요).

4.2 docker-compose

```
# infra/docker-compose.yml
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["80:80", "443:443"]
    volumes:
      - ./nginx/conf.d:/etc/nginx/conf.d:ro
      - ./certbot/conf:/etc/letsencrypt:ro
    depends_on: [api, web]

  api:
    image: ghcr.io/byeori/byeori-api:${TAG:-latest}
    env_file: [./env/.env]
    depends_on:
      postgres: { condition: service_healthy }
      minio: { condition: service_started }
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/actuator/health"]
      interval: 15s
      timeout: 5s
      retries: 5

  web:
    image: ghcr.io/byeori/byeori-web:${TAG:-latest}
    env_file: [./env/.env]
    depends_on: [api]

  postgres:
    image: postgres:16-alpine
    env_file: [./env/.env]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5

  minio:
    image: minio/minio:latest
    command: server /data --console-address ":9001"
    env_file: [./env/.env]
    volumes: [miniodata:/data]

volumes:
  pgdata:
  miniodata:
```

Flyway는 `api` 부팅 시 `db/migration` 을 자동 적용한다.

4.3 nginx · TLS

```
# infra/nginx/conf.d/byeori.conf
server {
    listen 443 ssl;
    server_name byeori.example.com;
    ssl_certificate      /etc/letsencrypt/live/byeori.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/byeori.example.com/privkey.pem;

    gzip on;
    gzip_types application/json text/css application/javascript;

    location /api/ { proxy_pass http://api:8080; proxy_set_header Host $host; }
    location /      { proxy_pass http://web:3000; proxy_set_header Host $host; }
}
```

TLS 인증서는 Let's Encrypt(certbot)로 발급·자동 갱신.

4.4 환경변수 (`infra/env/.env.example`)

```
# --- DB ---
POSTGRES_DB=byeori
POSTGRES_USER=byeori
POSTGRES_PASSWORD=__change_me__
SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/byeori
# --- JWT ---
JWT_ACCESS_SECRET=__change_me__
JWT_REFRESH_SECRET=__change_me__
JWT_ACCESS_EXP_MIN=30
JWT_REFRESH_EXP_DAYS=14
# --- 소셜 OAuth ---
KAKAO_REST_API_KEY=
NAVER_CLIENT_ID=
NAVER_CLIENT_SECRET=
GOOGLE_CLIENT_ID=
APPLE_CLIENT_ID=
# --- 공공데이터 / 지도 (백엔드 전용) ---
KOPIS_API_KEY=          # 1인 1개·PC(서버) 전용, 앱 직접호출 금지
TOURAPI_KEY=            # 한국관광공사 TourAPI 4.0 / 문화포털
KAKAO_MAP_REST_KEY=
# --- 스토리지 ---
MINIO_ROOT_USER=byeori
MINIO_ROOT_PASSWORD=__change_me__
MINIO_BUCKET=byeori-media
# --- 회원 탈퇴 유예 ---
WITHDRAWAL_GRACE_MONTHS=3
```

4.5 스케줄 배치 (@Scheduled)

배치	주기	내용	근거
KOPIS 동기화	매일 1회	진행중·예정 공연/시설 갱신. 1초당 10회 throttle , venue→performance 순, 성공 시에만 synced_at 갱신	KOPIS 속도 제한
TourAPI 동기화	매일 1회	음식점·관광지·문화시설(venues) / 축제(performances) 갱신, tour_content_id upsert	공모전 필수 활용
탈퇴 영구삭제	매일 1회	status=WITHDRAWN + withdrawn_at 경과(WITHDRAWAL_GRACE_MONTHS) 회원의 본인·리뷰·찜·일지 영구 삭제	soft delete 유예

모든 배치 결과는 **sync_logs** (동기화) 또는 애플리케이션 로그에 기록한다.

4.6 CI/CD (GitHub Actions → SSH 배포)

```
# .github/workflows/deploy.yml (요약)
on: { push: { branches: [main] } }
jobs:
  build-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build & push images
        run: |
          docker build -t ghcr.io/byeori/byeori-api:${{ github.sha }} apps/api
          docker build -t ghcr.io/byeori/byeori-web:${{ github.sha }} apps/web
          docker push ghcr.io/byeori/byeori-api:${{ github.sha }}
          docker push ghcr.io/byeori/byeori-web:${{ github.sha }}
      - name: Deploy over SSH
        run: |
          ssh $VM "cd /srv/byeori/infra && TAG=${{ github.sha }} docker compose pull && \
            TAG=${{ github.sha }} docker compose up -d"
```

4.7 백업 · 운영

- **DB 백업:** **scripts/backup.sh** 가 **pg_dump** 를 cron(매일)으로 실행, 보관 N일.
- **MinIO:** 버킷 데이터는 볼륨(**miniodata**) 백업 대상에 포함.
- **헬스체크:** **api** 는 **/actuator/health** , **postgres** 는 **pg_isready** .

4.8 공공데이터 이용 의무 체크리스트

위반 시 서비스 중단 사유가 되므로 배포 전 필수 확인.

- ☐ `source=KOPIS` 데이터 노출 화면에 **출처 문구 표기** (예: `출처: (재)예술경영지원센터 공연예술통합전산망, www.kopis.or.kr`)
- ☐ `source=TOURAPI` 데이터 노출 화면에 **한국관광공사 출처 표기**
- ☐ KOPIS 인증키는 **백엔드 전용** — 앱/웹에서 직접 호출 금지
- ☐ KOPIS 호출 **1초당 10회 throttle** 적용 확인
- ☐ 동기화 순서 **venue → performance** 준수(FK NOT NULL)

4.9 로컬 개발 시작

```
cp infra/env/.env.example infra/env/.env # 값 채우기
cd infra
docker compose -f docker-compose.yml -f docker-compose.override.yml up -d
# api:8080 · web:3000 · postgres:5432 · minio:9000(콘솔 9001)
```

앱(Expo)은 `apps/app` 에서 `npx expo start` , 웹은 `apps/web` 에서 `npm run dev` 로 기동한다.